The Sahyadri pkdgrav3 pipeline on pawna: setup, bugs fixed, and systematic tests of the halo catalogs

Sahyadri collaboration — prepared with Claude Code

11 November 2026, updated 12 November 2026

repository `a-paranjape/sahyadri-sandbox`, branch `rockstar-pkc-reader`

Abstract

We describe the end-to-end pipeline that runs a Sahyadri simulation on the pawna cluster: pkdgrav3 writing compressed `.pkc` snapshots, Rockstar reading them directly, consistent-trees, value-added catalogs (`.vahc`, `.tens`) and conversion to FITS, submitted as a chain of PBS jobs from one configuration file. We list the bugs found and fixed on the way, three of which change or blocked existing data products, and report systematic tests of the different ways of producing a halo catalog from the same simulation: the native `.pkc` reader, the same particles converted to Gadget format, the uncompressed (raw) pkdgrav3 particles, a 4% particle subsample, and repeated runs. With one reader the `.pkc` route reproduces the Gadget route byte for byte. Compression changes halo counts, masses, $V_{\rm max}$, positions and velocities at the per-mille level without bias; the inner structure (scale radius, spin) of halos with fewer than ∼1000 particles is the most sensitive. Wall time and peak memory of every stage are measured for 128^3 and 512^3 runs. All figures are made by a notebook from data files kept in the repository. The update of 12 November adds `vahc_fast.py`, which gives `.vahc`/`.tens` identical to the original code 12.6 times faster, the mean particle potential around halos (`.vpot`) with its checks, a test of running Rockstar in batches during the simulation, and the first measurements of the 1024^3 production run.

Contents

1 Overview **2**

2 Setting up and running the pipeline **2**
2.1 Dependencies and one-time setup . . . 2
2.2 One configuration file per run . . . 3
2.3 Submission, chaining and restarts . . . 3
2.4 Directory layout . . . 3

3 The compressed snapshot format and its precision **3**

4 Bugs found and fixed **3**

5 Systematic tests: different ways of making the halo catalog **5**
5.1 Set-up . . . 5
5.2 Exactness of the reader . . . 5
5.3 Effect of compression: raw particles vs `.pkc` . . . 5
5.4 Halo finding on a 4% subsample . . . 7
5.5 Merger trees . . . 7

6 Wall time and memory **7**

7 Faster value-added catalogs and the halo potential (`vahc_fast.py`, `.vpot`) **10**

8 Running Rockstar in batches during the simulation **12**

9 Toward production: consistent-trees resources and the first 1024^3 run **13**

10 Caveats and open questions **13**

11 Reproducing this report **13**

1 Overview

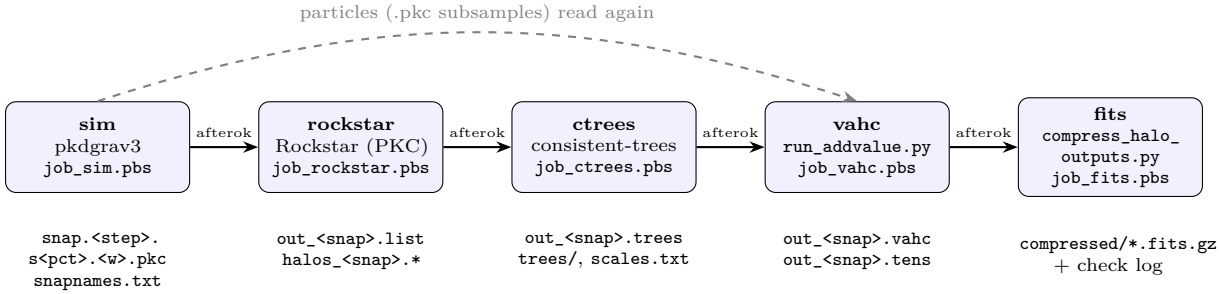


Figure 1: The five stages, each a PBS job started only if the previous one succeeded (`afterok`). One bash configuration file describes a run; `submit_pipeline.sh` submits the chain.

The stages and what each needs are listed in Table 1. The scripts are in `scripts/pkdgrav3_pipeline/`; the step-by-step guide (setup, all options, troubleshooting) is `docs/pkdgrav3_pipeline_pawna.md`, and the design and validation notes of the Rockstar reader are in `docs/rockstar_pkc_reader_plan.md`. This report summarises both and adds the systematic tests and resource measurements.

Stage	Code (version)	Reads	Writes
sim	pkdgrav3, `shadaba/pkdgrav3 main`	config → `p.par`, transfer function	`.pkc` files per output, subsample and writer; `snapnames.txt`; `pkc_check` per output
rockstar	Rockstar `36ce9eea36ee` + Sahyadri files (PKC reader, curvature, w_0–w_a)	`.pkc` (`FILE_FORMAT = "PKC"`); cosmology from the headers	`out_<snap>.list`, `halos_<snap>.*`, `rockstar.cfg`
ctrees	consistent-trees `7c20add` + Sahyadri background patch; `clean_trees.pl`	all `out_<snap>.list`	`out_<snap>.trees` (hlists), `trees/`, `../scales.txt`, configs in `logs/`
vahc	`scripts/post-process/run_addvalue.py` (`vahc_fast.py`; `AddValue` with `PP_FAST=0`)	particles (CIC density; potential for `.vpot`), `out_<snap>.trees`	`out_<snap>.vahc` (b_1, tidal and Hessian eigenvalues), `out_<snap>.tens`, `out_<snap>.vpot`
fits	`compress_halo_outputs.py`, `check_compressed_halos.py`	`.trees`, `.vahc`, `.tens`, `trees/`	`compressed/*.fits.gz` with cosmology and run metadata; column-by-column check

Table 1: Pipeline stages. The order is forced: `.vahc`/`.tens` rows are the halos of `out_<snap>.trees` (consistent-trees IDs), and the FITS stage folds b_1 into the basic catalog and may delete the ASCII files.

2 Setting up and running the pipeline

2.1 Dependencies and one-time setup

Everything is built as the user, in directories on `/storage`; nothing needs root.

pkdgrav3 with the compressed output, built by pkdgrav3's own `tools/pawna/setup_pawna.sh` (GCC 11, CMake 3.22, OpenMPI 5 module, HDF5 1.10, GSL, FFTW 3, Boost from the system; zstd, cpptrace and libdwarf fetched by CMake). It also provides `pkc_check`, `pkc_to_gadget` and the zstd library that Rockstar links against.

Halo codes and Python `scripts/pkdgrav3_pipeline/setup_pawna.sh <CODES> [PKD_IO_ROOT]` clones Rockstar (bitbucket, `36ce9eea36ee`) and consistent-trees (`7c20add`), copies the changed files kept in this repository under `code/` over them, builds Rockstar with `make with_hdf5_pkc`, builds consistent-trees, creates a self-contained Python venv (numpy, scipy, pandas, h5py, fitsio, zstandard; the system scipy on pawna needs numpy < 1.25) and compiles the C helpers of the post-processing (`cic_update.so`, `ngp_update.so`). It took 40 minutes on the pawna login node, 37 of them pip building packages from source.

```
# pkdgrav3 (once, 20-40 min)
git clone -b main https://github.com/shadaba/pkdgrav3.git /tmp/$USER-pkd-setup
bash /tmp/$USER-pkd-setup/tools/pawna/setup_pawna.sh /storage/$USER/pkdgrav3_io
# this repository + Rockstar + consistent-trees + Python (once, ~40 min)
git clone -b rockstar-pkc-reader https://github.com/a-paranjape/sahyadri-sandbox.git
```

```
bash sahyadri-sandbox/scripts/pkdgrav3_pipeline/setup_pawna.sh /storage/$USER/sahyadri_codes /storage/$USER/
    pkdgrav3_io
```

2.2 One configuration file per run

A run is described by a bash `KEY=VALUE` file (`example_512.conf`, `example_test128.conf`): paths (`SANDBOX`, `PKD_IO_ROOT`, `CODES`, `SCRATCH`, `LOGS`); run identity (`STEM`, `REAL`, `NPART`, `LBOX`, `SEED`); cosmology (including w_0, w_a and $\Omega_{\rm DE}$); output redshifts and time steps; snapshot format (`.pkc` subsample percents, optional HDF5); per-stage cores, memory and wall time; Rockstar settings (readers, writers, `FORCE_RES` defaulting to the pkdgrav3 softening $L/N/50$, minimum halo sizes, `PKC_SUBSAMPLES`); tree cleaning; grid, downsampling and parallelism of the value-added step; FITS options. Every job sources the same file, so a run is reproducible from it.

2.3 Submission, chaining and restarts

```
P=sahyadri-sandbox/scripts/pkdgrav3_pipeline
$P/submit_pipeline.sh my.conf --dry-run        # print the qsub commands
$P/submit_pipeline.sh my.conf                  # sim -> rockstar -> ctrees -> vahc -> fits (afterok chain)
$P/submit_pipeline.sh my.conf --stages sim     # only the simulation now ...
$P/submit_pipeline.sh my.conf --stages rockstar,ctrees,vahc,fits --after 2536500.pawna   # ... the rest after
    it
$P/submit_pipeline.sh my.conf --hold           # first job held; qrls <id> starts the chain
for r in 1 2 3 4 5; do                         # realisations: one config (and one chain) each
  sed "s/^REAL=.*/REAL=$r/; s/^SEED=.*/SEED=$((1000+r))/" my.conf > my_r$r.conf; $P/submit_pipeline.sh my_r$r.
    conf
done
```

Each job writes its own log, `$LOGS/<stage>.<run>.<jobid>.log` (PBS's copy-back is unreliable on pawna), and a memory record `<...>.mem.tsv` (Section 6). If a stage fails, PBS removes the jobs that depend on it; after fixing the cause the chain is resubmitted from that stage with `-stages`. The vahc and fits stages skip finished snapshots and write under temporary names; sim, rockstar and ctrees start over (Rockstar can resume from its `restart.cfg` with `RS_RESTART=1`). Any stage also runs without PBS, e.g. `CONF=$PWD/my.conf PIPE_ENV=$P/pipeline_env.sh STAGE=vahc bash $P/job_vahc.pbs`.

2.4 Directory layout

Outputs follow the layout the post-processing (`readers.py`) expects: `$SCRATCH/sims/<STEM>/r<REAL>/` (parameter file, `.pkc` snapshots, `snapnames.txt`, logs), `$SCRATCH/halos/<STEM>/r<REAL>/` (Rockstar, consistent-trees, `.vahc`, `.tens`, `compressed/`) and `$SCRATCH/halos/<STEM>/scales.txt`. `snapnames.txt` maps Rockstar's snapshot number to pkdgrav3's step. For the value-added step only the `.pkc` subsamples selected by `PP_DOWN_TO` are read; with `PP_DOWN_TO=512` at 2048^3 these are the 1% and 3% files, so keeping them (4% of the snapshot volume) is enough to recompute `.vahc`/`.tens` later.

3 The compressed snapshot format and its precision

pkdgrav3 writes each output as `.pkc` files: particles are split into random subsamples (by default 1, 3, 5, 9, 10, 12, 15, 20, 25%), and within each file positions are stored as integers on a grid of `ncell`×`sub` steps per box side and velocities in steps of `vel_step`, then entropy coded with zstd. For the 512^3, $50\,h^{-1}$Mpc validation run the header gives `ncell` = 512, `sub` = 50: a position step of $50/(512 \cdot 50) = 1.953\,h^{-1}$kpc, equal to the force softening $L/N/50$, and a velocity step of 2.44 km/s, about 10 bytes per particle. Figure 2 compares the particles of the 1% subsample at $z = 0$ with pkdgrav3's raw (single precision, uncompressed HDF5) output of the same step: the errors are uniform within half a step, as expected for rounding.

4 Bugs found and fixed

Table 2 lists the problems found while building and testing the pipeline. The first three affect data products that already exist or could not be made with the code in the repository.

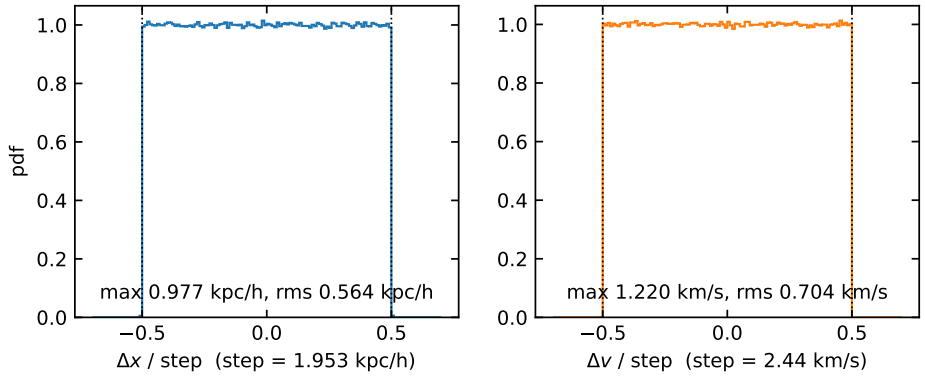


Figure 2: Per-particle difference between `.pkc` and the raw HDF5 output (512^3, $z = 0$, 1% subsample), in units of the quantization step. Dotted: ± half a step.

Where	Problem and effect	Fix
`correlations.density_hessian_field`	Smoothed the caller's Fourier-space density in place. `add_value` computes the Hessian at 3 then $5\,h^{-1}$Mpc, so **`lamH1/2/3_R5Mpch` in every existing `.vahc`** came from a field smoothed twice (3 and $5\,h^{-1}$Mpc Gaussians); `.tens` computed after a `.vahc` in the same process were affected the same way. On the 128^3 test the old values have 0.55–0.63 of the correct rms (Fig. 3); every other column is unchanged.	Work on a copy (as `tidal_tensor_field` already did).
`correlations.halobyhalo_b1`	Raised "Please specify number density nbar" since commit `e77922b` (Feb 2026): `.vahc` could not be produced with the repository code.	`nbar=inf` (no shot-noise subtraction, as in the production code); output byte-identical to production.
`postprocess.py` (repository version)	Sorted halos by mass before `add_value`, so `.vahc` rows did not follow `.trees`; production (`sorthalos=False`) did not.	Keep file order; add `add_tens`; the FITS converter refuses misaligned files.
`readers.read_block`	`down_to` was dead code after an early return: all particles were read and used; files not closed.	Downsampling restored; files closed.
`cic_update.c` (production copy on pegasus)	Particle count passed as `int`: overflow above 2^{31} particles (production stayed below with `down_to=512`).	Repository version uses `long`.
consistent-trees (upstream)	Flat ΛCDM everywhere: time table from Ω_m only, $\Delta_{\rm vir}$ with a cosmological constant, w_0/w_a in `merger_tree.cfg` ignored. Affects every curved or w_0–w_a run (times, accretion rates, consistency step).	Background $E(a)$ as in Rockstar's `hubble.c`, in C and Perl; flat ΛCDM byte-identical to upstream.
`gen_merger_cfg.pl`	w_0, w_a for consistent-trees taken from the hand-written Rockstar input config, not from the catalogs.	Read from a `\#w0` line Rockstar now prints (only for non-ΛCDM).
Rockstar	Not deterministic: `srand(getpid())` and particle arrival order with several readers.	Not changed; a fixed-seed test build is used for exact tests only.
Pipeline (pawna)	PBS `-o` copy-back unreliable; system scipy incompatible with current numpy; HDF5 file locking hangs h5py on `/storage` (NFS).	Jobs write their own logs; self-contained venv; `HDF5_USE_FILE_LOCKING=FALSE`.

Table 2: Bugs and problems found, their effect and the fix (details in `docs/pkdgrav3_pipeline_pawna.md`, Section 7, and `docs/rockstar_pkc_reader_plan.md`).

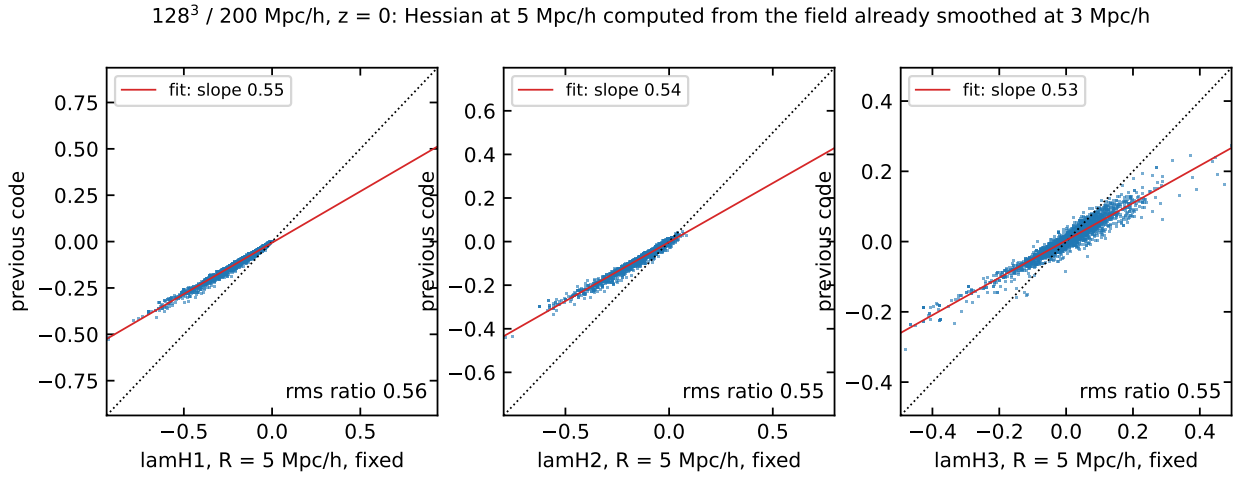


Figure 3: Density-Hessian eigenvalues at $R = 5\,h^{-1}$Mpc for the 4736 halos of the $128^3/200\,h^{-1}$Mpc test at $z = 0$: previous code against the fixed code. The previous values are a strongly smoothed version of the correct ones.

5 Systematic tests: different ways of making the halo catalog

5.1 Set-up

One pkdgrav3 run, 512^3 particles in $50\,h^{-1}$Mpc ($m_p = 8.11 \times 10^7\,h^{-1}M_\odot$, flat ΛCDM, outputs at $z = 3, 2, 1, 0.5, 0.25, 0$), was written both as `.pkc` (default precision) and as standard HDF5. Rockstar (same binary and settings, 8 readers and 8 writers, `FORCE_RES` $= 1.95\,h^{-1}$kpc, `MIN_HALO_OUTPUT_SIZE` $= 20$) was run on the inputs of Table 3, all six outputs in one invocation (temporal halo finding on).

Route	Input to Rockstar
`pkc`	the `.pkc` files, native reader (`FILE_FORMAT = "PKC"`) — the pipeline's route
`pkc2`	the same, run again: Rockstar's own run-to-run scatter (the noise floor)
`gad`	the same quantized particles converted to Gadget-2 by `pkc_to_gadget -blocks 8`
`h5g` ("raw")	pkdgrav3's uncompressed HDF5 output of the same steps, converted to Gadget-2 (`-from-hdf5`)
`pkcsub`	native reader on the 1% and 3% subsamples only (`PKC_SUBSAMPLES = "1,3"`): a quick-look option, particle mass ×25
`pkc1`, `gad1`	$z = 0$ only, one reader, fixed-seed test build: exactness of the reader

Table 3: Rockstar input routes compared.

Halos of `out_<snap>.list` were matched between two routes by nearest centre (periodic KD-tree), within $10\,h^{-1}$kpc below $M_{\rm vir} = 10^{12}\,h^{-1}M_\odot$ and $50\,h^{-1}$kpc above. A matched pair whose masses differ by more than 50% is a host/subhalo assignment "flip"; flips are counted but excluded from the medians.

5.2 Exactness of the reader

With one reader and the fixed seed, `pkc1` and `gad1` give byte-identical `out_0.list`, `halos_<snap>.0.ascii` and `halos_<snap>.0.bin` (checked with `cmp` in validation jobs 2536453 and 2536496). The native reader therefore hands Rockstar exactly the particles, masses and header values that the Gadget route does. With 8 readers the catalogs of `pkc` and `gad` differ, but only as much as two `pkc` runs differ from each other (Fig. 5): what remains is Rockstar's dependence on the order in which particles reach the writers.

5.3 Effect of compression: raw particles vs `.pkc`

Table 4 and Figs. 4–7 give the effect of the `.pkc` quantization. The halo counts agree with the raw-particle route to within the Poisson error in every mass bin and at every output, with no mass bias: the signed median $\Delta M/M$ is zero, because for most halos the same particles are bound in both routes. In absolute terms, $M_{\rm vir}$ and $V_{\rm max}$ change by 4–5 $\times\,10^{-3}$ (median) for 10^{10}–$10^{11}\,h^{-1}M_\odot$ (123–1230 particles), $\approx 10^{-3}$ at 10^{11}–10^{12} and $\lesssim 5 \times 10^{-4}$ above. Rockstar is deterministic for a given input (`pkc` vs `pkc2` medians are zero or $\sim 10^{-4}$), so these differences are entirely due to the quantization, not to noise. Centres move by 0.6–0.9 h^{-1}kpc, i.e. by about the rms quantization error of one particle ($0.56\,h^{-1}$kpc) and well below the softening ($1.95\,h^{-1}$kpc).

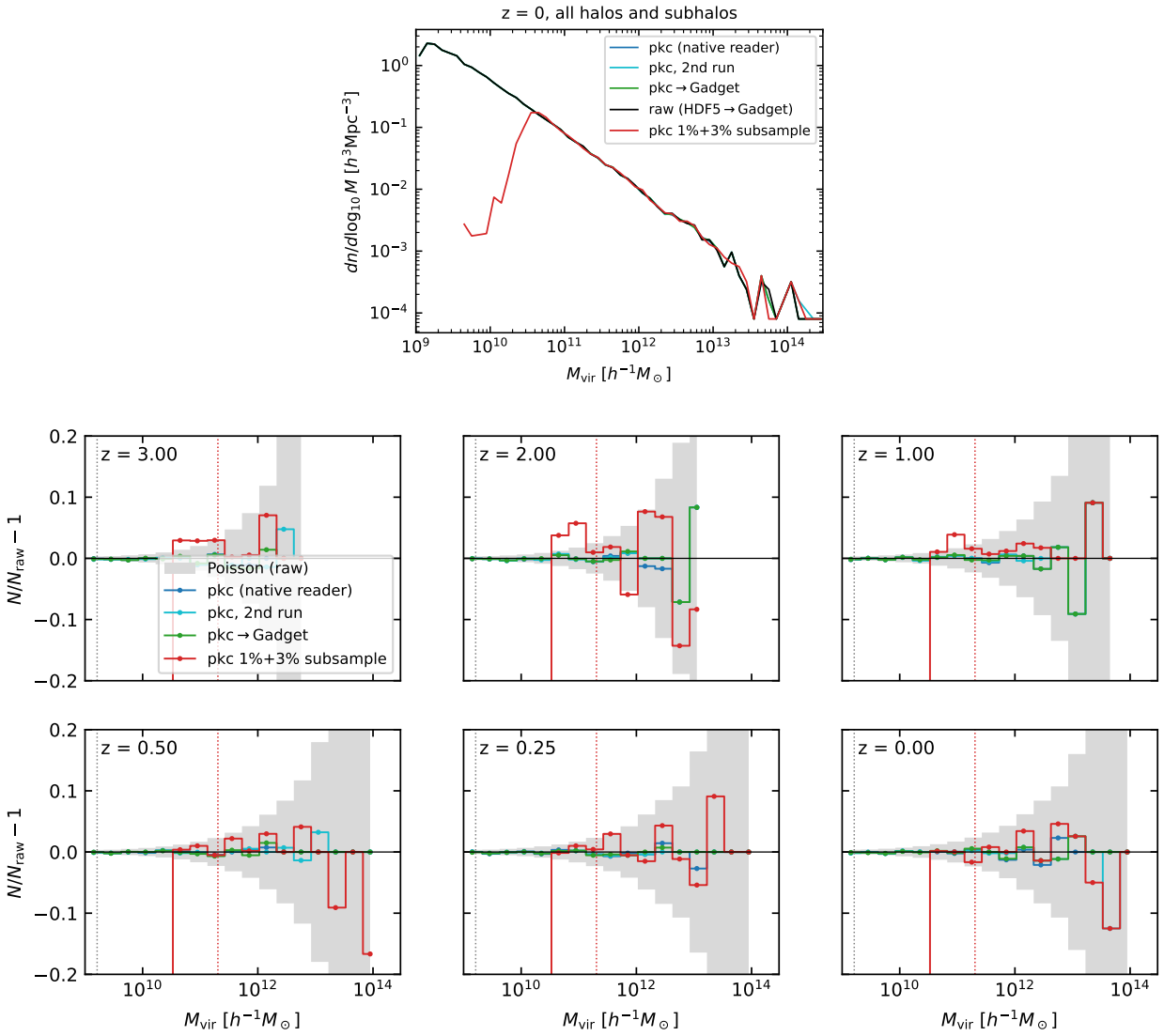


Figure 4: Top: mass function at $z = 0$ for each route. Bottom: ratio to the raw-particle route at every output; grey band: Poisson error of the raw counts; grey dotted line: 20 particles; red dotted line: 100 particles of the subsample route.

$\log_{10} M_{\rm vir}$ $[h^{-1}M_\odot]$	N	$N/N_{\rm raw} - 1$ [%]	matched [%]	flips [%]	med $\Delta M/M$	med $\lvert\Delta M/M\rvert$	med $\lvert\Delta V_{\rm max}/V_{\rm max}\rvert$	centre [kpc/h]	$\lvert\Delta R_s/R_s\rvert$ [%], bias	$\lvert\Delta\lambda/\lambda\rvert$ [%], bias
10–11	31764	-0.00 (0.6)	99.50 (99.92)	0.9 (0.2)	0	3.9e-3 (0)	5.1e-3 (0)	0.86 (0.00)	5.52 (0.00), +0.42	3.80 (0.00), -0.31
11–12	4213	-0.12 (1.5)	99.69 (99.93)	1.3 (0.2)	0	1.1e-3 (0)	1.2e-3 (6.0e-5)	0.64 (0.03)	1.73 (0.13), +0.07	1.09 (0.10), -0.06
12–13	510	-0.20 (4.4)	99.80 (100.00)	3.9 (1.0)	0	4.5e-4 (1.4e-4)	3.4e-4 (1.1e-4)	0.66 (0.16)	0.36 (0.12), -0.02	0.33 (0.11), -0.04
13–15	57	+1.79 (13.4)	100.00 (100.00)	5.3 (1.8)	-7.7e-5	2.7e-4 (8.7e-5)	1.3e-4 (5.7e-5)	0.87 (0.33)	0.11 (0.05), -0.03	0.15 (0.04), -0.01

Table 4: Compression effect at $z = 0$ (`pkc` vs raw), in mass bins of 10^{10}–10^{11} (123–1230 particles) up to $> 10^{13}\,h^{-1}M_\odot$; in brackets the same quantity for `pkc` vs `pkc2` (Rockstar noise) or, for $N/N_{\rm raw} - 1$, the Poisson error. Medians over matched halos without flips. Generated by the notebook.

The scale radius R_s and spin λ, which depend on the inner particles, are most affected: 5.5% and 3.8% median scatter below $10^{11}\,h^{-1}M_\odot$, with small biases (R_s +0.4%, λ −0.3%), falling to $\sim 1.7\%$ and 1.1% at 10^{11}–10^{12} and $< 0.4\%$ above $10^{12}\,h^{-1}M_\odot$. About 0.5% of the lowest-mass halos have no counterpart within $10\,h^{-1}$kpc, and 1–5% of the matched pairs are host/subhalo flips (against 0.2–2% between two Rockstar runs). The differences grow mildly with redshift (Fig. 6) but stay at the same order of magnitude. For quantities other than R_s and λ of halos below ~ 300 particles, the compression is negligible compared with Poisson noise in any statistic.

5.4 Halo finding on a 4% subsample

The `pkcsub` route runs Rockstar on the 1% and 3% subsamples together (4% of the particles, particle mass $\times 25$, so $10^{11}\,h^{-1}M_\odot$ is ≈ 49 particles). It takes 105 s and 0.84 GB instead of ≈ 800 s and 8.3 GB (Table 5). At $z = 0$ (Fig. 8), 83% of the subsample halos with 10^{11}–$10^{11.5}\,h^{-1}M_\odot$ have a counterpart in the full catalog, 92% at $10^{11.5}$–10^{12} and $\geq 99.6\%$ above $10^{12}\,h^{-1}M_\odot$. Masses are biased high by 4% at 10^{11}–$10^{11.5}$ (median $|\Delta M/M| = 10\%$), 1% (5%) at $10^{11.5}$–10^{12} and $< 1\%$ (3%) above; $V_{\rm max}$ is biased high by 8%, 3% and 2% respectively. Below ~ 100 subsample particles ($2 \times 10^{11}\,h^{-1}M_\odot$) the mass function is incomplete (Fig. 4, red line). The subsample route is therefore useful as a quick look at halos above $\sim 10^{12}\,h^{-1}M_\odot$ ($\gtrsim 500$ subsample particles), not as a replacement for the full catalog.

5.5 Merger trees

consistent-trees was run on the six outputs of the `pkc` and `h5g` routes (job 2536496). At $z = 0$ the hlists contain 181 621 hosts and 30 753 subhalos for `pkc`, against 181 707 and 30 873 for the raw particles: −0.05% and −0.4%, within the Poisson errors (0.2% and 0.6%). The host and subhalo mass functions agree in every bin within the Poisson error (Fig. 9). The wall times (643 and 452 s for identical work) reflect load on the shared `/storage` file system rather than the input.

6 Wall time and memory

pawna puts every PBS job into the same cgroup (`/system.slice/pbs.service`), so the kernel keeps no per-job memory peak, and PBS's own accounting was not kept after the jobs ended. Every pipeline job therefore runs `memmon.sh`, which every 5 s (2 s for the validation steps) sums over the job's process tree the proportional set size (PSS: a page shared by n processes counts $1/n$ in each, so the sum is the memory really occupied) and the resident set size (RSS, shared pages counted in every process: an upper bound). Peaks shorter than the sampling interval can be missed. The monitor was checked on the login node against two processes allocating 2 GB and 1 GB (measured 3.04 GB PSS).

Table 5 and Fig. 10 give the results; all runs used the pawna configurations of Section 2 (`SIM_NCPU=64` for the 512^3 simulation).

- **Simulation.** 25.1 GB for 512^3 on 64 cores (80 min), 5.9 GB for 128^3. Memory is almost constant during the run (Fig. 10); 64 times the particles cost only 4.3 times the memory, so at 128^3 a fixed overhead dominates. The single-process `maxrss` reported by `/usr/bin/time` agrees with the PSS sum.
- **Rockstar.** 8.0 GB with 27 processes (8 readers, writers and the server) for the 512^3 run of the pipeline and 8.2–8.4 GB for each full route of the validation; no single process exceeds 0.45 GB, so the memory is spread over the writers. The total is 1.9 times Rockstar's particle array ($512^3 \times 32\,\mathrm{B} = 4.3$ GB). The input route makes no difference: the native reader, the Gadget files and the raw HDF5 data all need 8.2–8.4 GB and 800–850 s, while the Gadget routes need a conversion step on top (388 s, 10.0 GB). One reader (`pkc1`, `gad1`) needs 6.5 GB.
- **consistent-trees.** 0.8 GB in the pipeline and 1.4 GB in the validation job ; 8–11 min.
- **`.vahc` (`run_addvalue`).** The most memory-hungry stage: 91.6 GB at 512^3 (61 min), 2.3 GB at 128^3. The memory is set by the density grid, not by the particles: each process holds the complex grids of the tensor calculation, about $12 \times 16 \times$`PP_GRID`^{3} bytes (25.8 GB for `PP_GRID=512`, measured largest process 25.5 GB; 0.4 GB for the 128^3 run's grid of 128), and `PP_NPROC` snapshots run at once (4 at 512^3, so 70–90 GB for the first 30 min, then ≈ 40 GB for the last two of the six snapshots; Fig. 10). This part does not grow with the resolution of the simulation: at the same `PP_GRID`, a 1024^3 or 2048^3 run needs the same ≈ 26 GB per process as measured here. The particles add almost nothing either: without downsampling the density is built block by block from the `.pkc` files (byte-identical `.vahc`/`.tens` to loading all positions at once, same time). Only a larger grid raises it, as `PP_GRID`^{3}: ≈ 87 GB per process for the grid of 768 that production used at 2048^3 (`PP_NPROC` $\leq$ 6–7 on one 755 GB node).

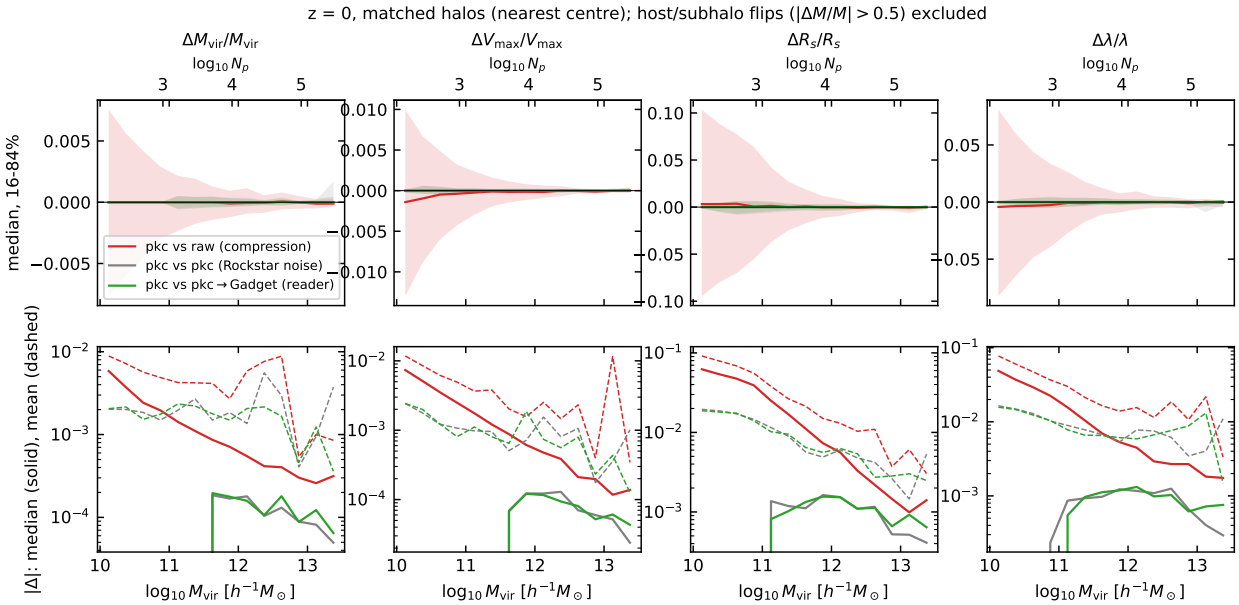


Figure 5: Differences between matched halos at $z = 0$ as a function of mass (top axis: number of particles). Top row: median and 16–84% range of the signed relative difference; bottom row: median absolute difference. Red: `.pkc` vs raw particles (effect of compression); grey: two runs on the same `.pkc` input (Rockstar noise); green: native reader vs the same particles in Gadget format.

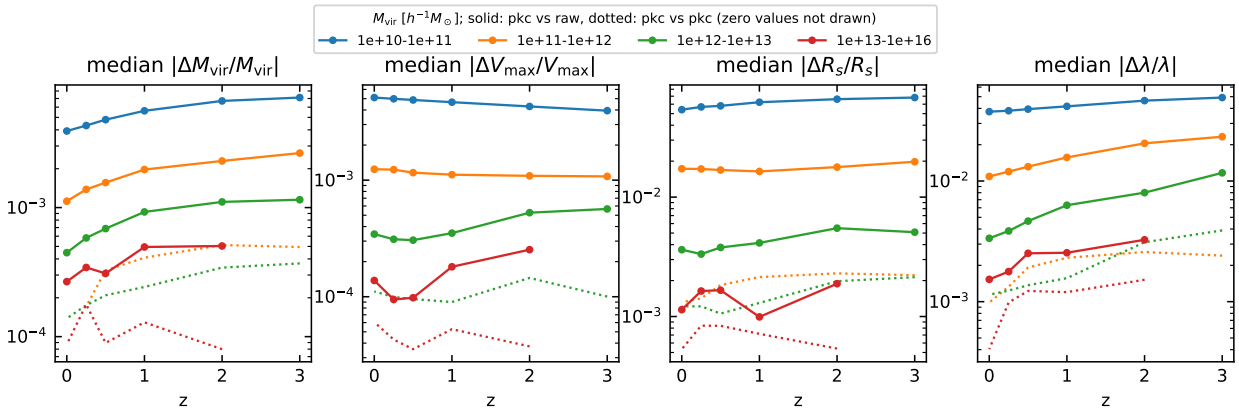


Figure 6: Median absolute difference vs redshift in four mass bins. Solid: `.pkc` vs raw; dotted: `pkc` vs `pkc2` (values of exactly zero are drawn at 10^{-6}).

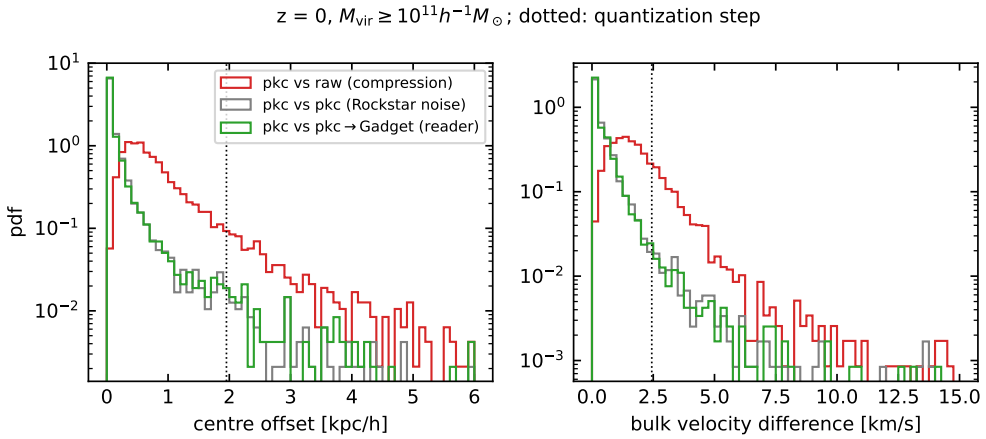


Figure 7: Distribution of centre offsets and bulk-velocity differences of matched halos with $M_{\rm vir} \geq 10^{11}\,h^{-1}M_\odot$ at $z = 0$.

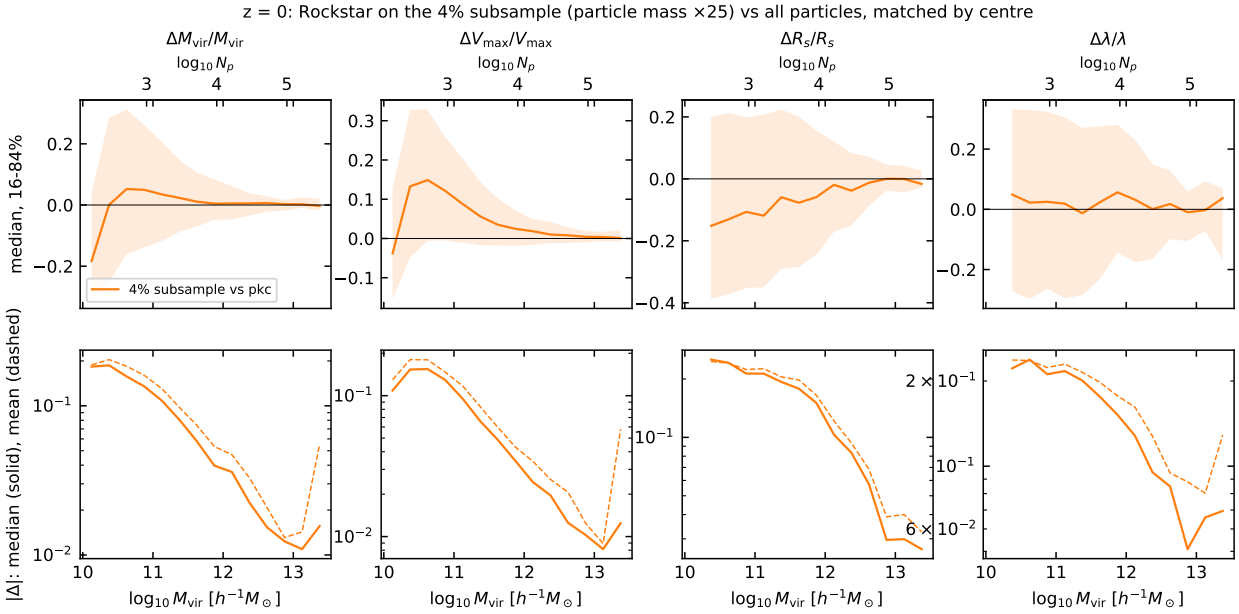


Figure 8: Rockstar on the 4% subsample vs on all particles at $z = 0$, matched by centre; as Fig. 5 (solid: median, dashed: mean absolute difference). Top axis: number of subsample particles.

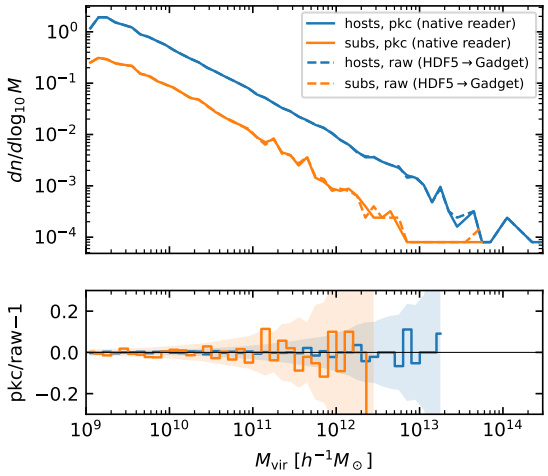


Figure 9: Host and subhalo mass functions of the consistent-trees hlists at $z = 0$ for `.pkc` input and raw particles, and their ratio with Poisson bands.

- **Fits.** 5.9 GB at 512^3, 0.1 GB at 128^3; minutes.

Extrapolating to Sahyadri's 2048^3 (64 times the 512^3 particle number), and keeping in mind that only two sizes were measured: the simulation needs at most ~ 1.6 TB in total if memory scaled with the particle number (spread over several nodes); Rockstar needs about twice its particle array, i.e. ~ 0.53 TB of a node's 755 GB if that ratio holds, which should be checked with a 1024^3 run; consistent-trees and the FITS stage are small; `run_addvalue` does not grow with resolution at fixed `PP_GRID` and is set by `PP_GRID` and `PP_NPROC` as above.

Run	Stage	Wall [s]	PSS [GB]	RSS [GB]	proc.
512^3	sim	4780	25.09	25.10	4
512^3	rockstar	475	8.03	8.22	27
512^3	ctrees	506	0.80	0.91	9
512^3	vahc	3666	91.56	91.74	9
512^3	fits	153	5.90	5.90	2
128^3	sim	110	5.86	5.88	4
128^3	rockstar	41	0.20	0.38	27
128^3	ctrees	9	0.00	0.01	2
128^3	vahc	49	2.29	2.52	10
128^3	fits	6	0.10	0.11	2

Step (512^3 validation)	Wall [s]	PSS [GB]	RSS [GB]	halos z=0
pkc	803	8.33	8.86	239939
pkc2	849	8.35	8.88	239965
gad	818	8.25	8.78	239946
h5g	815	8.23	8.75	239973
pkc1	764	6.53	6.54	239997
gad1	798	6.53	6.54	239997
pkcsub	105	0.84	0.95	15851
convert	388	10.04	10.05	–
ctrees pkc	643	1.44	1.44	–
ctrees h5g	452	1.43	1.44	–

Table 5: Left: pipeline stages for the 128^3 and 512^3 runs (both $200\,h^{-1}$Mpc). Right: Rockstar input routes and other steps of the $512^3/50\,h^{-1}$Mpc validation (6 outputs; `pkc1`/`gad1`: $z = 0$ only). PSS and RSS are peaks of the sum over the job's processes; "proc." is the number of processes at the PSS peak.

7 Faster value-added catalogs and the halo potential (`vahc_fast.py`, `.vpot`)

`scripts/post-process/vahc_fast.py` (class `FastAddValue`, a subclass of `AddValue`) computes `.vahc` and `.tens` in one pass: one real FFT of the density, a separable Gaussian kernel, one inverse real FFT per tensor component and smoothing scale on `-threads` threads (`scipy.fft`), one batched `eigvalsh` and vectorized interpolation for all halos, and b_1 as a single filtered field read at the 8 CIC corners of each halo instead of a sum over modes per halo. `run_addvalue.py -fast` selects it; `PP_FAST=1` is now the pipeline default (`PP_FAST=0` restores `AddValue`).

Same results. Old and new code were run in one process on the same density and halos (`test_vahc_fast.py`): every `.vahc` and `.tens` column is identical in float32 and the halo IDs are the same, for 128^3 (snapshot 4, grid 128, job 2536678) and for 512^3 (`lcdm512` $z = 0$, grid 512, 234 258 halos, job 2536679). The text output formats were also compared (0 of 200 004 float32 strings differ).

Time. For the 512^3 snapshot with 8 FFT threads: 147 s for both files against 1126 s (`.vahc`) + 722 s (`.tens`) for `AddValue`, 12.6 times faster. Of the 147 s the tidal FFTs (33 scales × 6 inverse FFTs) take 123.5 s, eigenvalues 12.7 s, Hessian 6.6 s, b_1 1.7 s, the forward FFT 0.9 s; reading the `.pkc` snapshot and building the density takes another 24 s for both versions. In the pipeline (Table 6) a snapshot takes 4.5–5 min with 4–5 threads per snapshot. The cost is set by the grid, not the number of halos, so it should not grow much at 1024^3 and 2048^3; there reading the particles ($\sim$35 min per 2048^3 snapshot from `/storage`, estimated) becomes the largest part.

Memory. 12–13 GB per process at grid 512 (about $6 \times 16 \times$`PP_GRID`^{3} bytes, half of `AddValue`'s), measured as job PSS divided by the number of snapshot processes (Table 6).

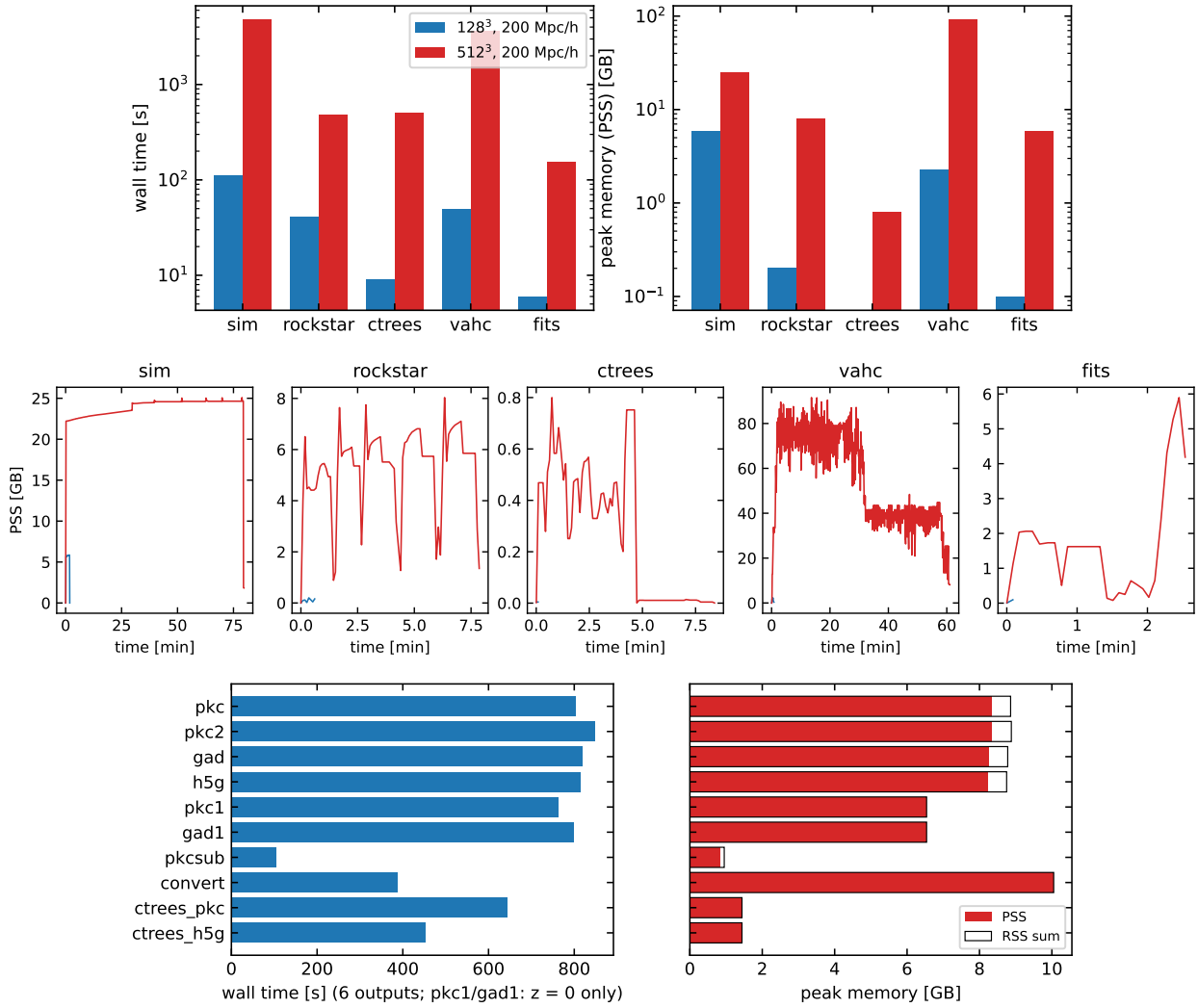


Figure 10: Top: wall time and peak memory per pipeline stage. Middle: memory (PSS) during each stage. Bottom: Rockstar input routes and other validation steps at 512^3.

`.vpot`: the mean particle potential around halos (`-pot`, `PP_POT=1`). For snapshots whose `.pkc` files store the pkdgrav3 potential ($z \le 0.5$ in production), `out_<snap>.vpot` has, for every halo of `out_<snap>.trees`, the number of particles and their mean potential in $(\mathrm{km/s})^2$ within spheres of radius r_s, R_{200b} (from M_{200b}), 0.5, 1 and 1.5 R_{vir}, and the Gaussian-weighted mean potential at the seven `.vahc` smoothing scales. pkdgrav3's softened self potential $-(105/64)\, m/\epsilon$, the same for every particle, is removed. The particles are read once, in the same pass as the density: the sphere sums run in C with OpenMP (`vpot_sums.c`), the grid scales use a CIC grid of the potential smoothed with the `.vahc` kernel and divided by the smoothed density. The FITS stage appends the columns to `out_<snap>_vahc.fits.gz` (card `HAS_VPOT`).

Checks of `.vpot`.

- *Spheres:* for 300 random halos per sphere the counts and mean potentials equal a KD-tree sum over all particles (`scipy cKDTree`, periodic): 0 count and 0 mean mismatches at all five radii, worst relative difference 5×10^{-8} (float32), at 128^3 snapshots 70 and 100 and 512^3 snapshot 5. Median particle numbers at 128^3: 2 (r_s), 25 ($0.5R_{\mathrm{vir}}$), 45 (R_{vir}), 48–50 (R_{200b}), 55–58 ($1.5R_{\mathrm{vir}}$). The C routine alone was also checked against the KD-tree for 2000 random spheres × 5 radii, including spheres across the box edge and zero and undefined radii (0 mismatches).
- *Grid scales:* against direct Gaussian-weighted sums over the particles around the same grid point (40 halos per scale): median relative difference $\le 10^{-5}$, maximum $\le 10^{-3}$ except one halo at $5\,h^{-1}$Mpc (1.2×10^{-2}), as expected from CIC.
- *Normalisation and self term:* the particle potential minus the self term was fitted against an FFT Poisson solution of the same particles (CIC on 4× the particle grid, deconvolved, the routine validated in the Gadget-4 comparison): slope 1.004 (128^3, $z = 0.38$), 1.007 (128^3, $z = 0$) and 1.002 (512^3, $z = 0$, grid 1024), correlation 0.998–0.9996, offset 0.2–1% of the rms potential. The self terms were -1.2×10^5, -9.0×10^4 and $-5.6 \times 10^3\,(\mathrm{km/s})^2$. The `.pkc` header field `time` is pkdgrav3's code time, not the scale factor; a must be taken from `redshift`.

The potential costs 47–68 s per snapshot on top of the tidal FFTs (grid 512, 4–5 threads); going from two to five spheres added about 11 s (Table 6).

Run (job)	Outputs	Snapshots	Wall	Per snapshot [s]	Job PSS [GB]	Notes
128^3, 101 outputs (2536685)	`.vahc`, `.tens`, `.vpot` (2 spheres)	85	3.96 h	200–250	45.9	4 proc. × 4 threads; vpot 50 s
128^3, same (2536697)	`.vpot` (5 spheres)	37	0.93 h	266–467	46.0	4 × 4; vpot 61–68 s
512^3, 6 outputs (2536691)	`.vahc`, `.tens`, `.vpot` (2 spheres)	6	5 min	258–273	78.4	6 × 5; vpot 47–51 s

Table 6: `vahc_fast.py` in the pipeline, grid 512. "Per snapshot" is the `add_value` time of one snapshot from the job log, without reading and density (14–41 s); the tidal FFTs are 160–220 s of it (396 s for one snapshot of job 2536697, which shared the node). PSS is the peak sum over the job's processes (`memmon.sh`).

8 Running Rockstar in batches during the simulation

Rockstar can process the snapshots in groups while the simulation is still running, so that only the last group is left when it ends. Rockstar skips loading the previous snapshot's halos (needed for the descendant links) only at `STARTING_SNAP`, so a batch of snapshots $k \ldots k{+}K{-}1$ is run with `STARTING_SNAP = 0`, `RESTART_SNAP = k` and a `SNAPSHOT_NAMES` file holding the first $k{+}K$ names (`NUM_SNAPS` is taken from its length). Test: `/storage/shadab/pipeline_test/rsbatch/rs_batch_test.pbs`, comparison `compare_lists.py`. Rockstar is not reproducible run to run, so a batched run is compared with a single run and two single runs with each other: halos are matched by position ($0.05\,h^{-1}$Mpc) and mass (1%), and the descendant IDs of matched halos are compared.

Run	Batch size K	Descendant agreement single vs single	Descendant agreement single vs batched
$512^3/50\,h^{-1}$Mpc, 6 outputs	2	0.90373	0.90368
$128^3/200\,h^{-1}$Mpc, 101 outputs	10	0.89209	0.89235

Batched runs agree with a single run as well as two single runs agree with each other; halo counts differ by ∼0.1% in both comparisons, and the snapshots at batch boundaries get their descendants. Automatic submission of batches by the pipeline is not implemented yet.

9 Toward production: consistent-trees resources and the first 1024^3 run

- **consistent-trees** is serial (one process, plus a forked writer), so it now asks for 2 cores and can share a node: 16 GB/12 h up to 256^3, 32 GB/24–48 h up to 1024^3, 128 GB/168 h above (`setup_production.py`). Measured: 195 s and 0.02 GB for the 101-output 128^3 run, 1967 s and 1.5 GB for 6 outputs at 512^3.
- **FITS stage fixes** found with `.vpot`: the write-time read-back checksum now counts NaNs (a float sum with a NaN never equals itself); when only the vahc file of a snapshot is (re)written the `.trees` file is still read (the scale-factor card and the row check needed it, `KeyError: 'trees'` before); the check script compares the `.vahc` columns of the vahc FITS file separately from the `.vpot` ones. After the fixes: 0 failures on both test runs.
- 1024^3 **production run** (`Sahyadri_L200N1024/default1024/r1`, 2126 steps, 101 outputs), started 12 November 08:57 on one node with 96 cores (job 2536567; reduced from 192 so that it could start). pkdgrav3 reports 110–112 GB resident (its own per-step statistics), against 744–746 GiB for 2048^3. Steps took 120–137 s up to step 48 and 70 s from step 56 on; the first output (step 124, $z = 12$, all subsamples 6.75 GiB) was written to the node's SSD in 9 min and copied to `/storage` in 516 s (13 MB/s). The rest of the chain (Rockstar, consistent-trees, `vahc_fast` with `.vpot`, FITS) is queued behind it and will give the first measured Rockstar memory above 512^3.

10 Caveats and open questions

- The first 1024^3 run is in progress (Section 9); its Rockstar stage will give the first measured Rockstar memory above 512^3. Rockstar memory at 2048^3 (64 times the 512^3 particles) is still extrapolated (≈ 0.53 TB from twice the particle array), not measured.
- The `.vahc`/`.tens` timings of `vahc_fast.py` at 1024^3 and 2048^3 are estimates.
- Rockstar's particle mass constant ($\rho_{\rm crit} = 2.77519737 \times 10^{11}$) differs from the one used by `pkc_to_gadget` and the PKC reader ($2.77536627 \times 10^{11}$) by 6×10^{-5}; Sahyadri should pick one.
- Rockstar treats radiation as curvature; consistent-trees' `clean_trees.pl` matches hlists by five characters of the scale factor, so outputs closer than $\Delta a = 0.001$ would collide.
- The `.vahc` files on pegasus have wrong `lamH*_R5Mpch` columns (Section 4) and should be regenerated (the 1% and 3% `.pkc` subsamples, or the downsampled Gadget particles, suffice).

11 Reproducing this report

- `docs/report/extract_data.py` (with `extract_job.pbs`) reduces the pawna catalogs and logs to the files in `docs/report/data/`.
- `docs/report/make_figures.ipynb` makes every figure and the generated tables from those files only (`build_notebook.py` holds its source).
- Validation jobs: `scripts/rockstar/pkc_validation/` (`run_val512.pbs`, `rockstar_val.pbs`, `compare_*.py`); pipeline runs: `scripts/pkdgrav3_pipeline/` with the configurations given in the text.